1. Implement Prims Algorithm for minimum spanning tree.

```c
#include <stdio.h>
#include <limits.h>

#define V 5

int minKey(int key[], int mstSet[])
{
  int min = INT_MAX;
  int minIndex = -1;

  for (int v = 0; v < V; v++)
  {
      if (mstSet[v] == 0 && key[v] < min)
      {
          min = key[v];
          minIndex = v;
      }
  }

  return minIndex;
}

void printMST(int parent[], int graph[V][V])
{
  int totalWeight = 0;

  printf("Edge \tWeight\n");

  for (int i = 1; i < V; i++)
  {
      printf("%d - %d \t%d\n",
             parent[i], i, graph[i][parent[i]]);

      totalWeight += graph[i][parent[i]];
  }

  printf("Total weight of MST = %d\n", totalWeight);
}

void primMST(int graph[V][V])
{
  int parent[V];
```

```
  int key[V];
  int mstSet[V];

  for (int i = 0; i < V; i++)
  {
      key[i] = INT_MAX;
      mstSet[i] = 0;
  }

  key[0] = 0;
  parent[0] = -1;

  for (int count = 0; count < V - 1; count++)
  {
      int u = minKey(key, mstSet);

      mstSet[u] = 1;

      for (int v = 0; v < V; v++)
      {
          if (graph[u][v] &&
              mstSet[v] == 0 &&
              graph[u][v] < key[v])
          {
              parent[v] = u;
              key[v] = graph[u][v];
          }
      }
  }

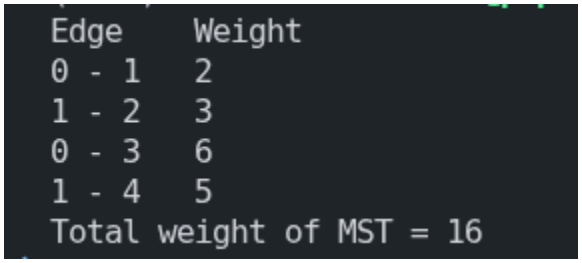
  printMST(parent, graph);
}


int main()
{
  int graph[V][V] = {
      {0, 2, 0, 6, 0},
      {2, 0, 3, 8, 5},
      {0, 3, 0, 0, 7},
      {6, 8, 0, 0, 9},
      {0, 5, 7, 9, 0}
  };

  primMST(graph);
```

```
  return 0;
}
```

Output:



2. Implement Dijkstra's Algorithm for Single Source Shortest Path.

```
#include <stdio.h>
#include <limits.h>

#define V 5

int minDistance(int dist[], int visited[])
{
  int min = INT_MAX;
  int minIndex = -1;

  for (int v = 0; v < V; v++)
  {
      if (visited[v] == 0 && dist[v] <= min)
      {
          min = dist[v];
          minIndex = v;
      }
  }

  return minIndex;
}
```

```c
void printSolution(int dist[])
{
  printf("Vertex \t Shortest Distance from Source\n");

  for (int i = 0; i < V; i++)
  {
      printf("%d \t %d\n", i, dist[i]);
  }
}

// Dijkstra's Algorithm
void dijkstra(int graph[V][V], int source)
{
  int dist[V];
  int visited[V];


  for (int i = 0; i < V; i++)
  {
      dist[i] = INT_MAX;
      visited[i] = 0;
  }


  dist[source] = 0;


  for (int count = 0; count < V - 1; count++)
  {
      int u = minDistance(dist, visited);

      visited[u] = 1;

      for (int v = 0; v < V; v++)
      {
          if (!visited[v] &&
              graph[u][v] != 0 &&
              dist[u] != INT_MAX &&
              dist[u] + graph[u][v] < dist[v])
          {
              dist[v] = dist[u] + graph[u][v];
          }
      }
  }
```

```
  printSolution(dist);
}


int main()
{
  int graph[V][V] = {
      {0, 10, 0, 5, 0},
      {10, 0, 1, 2, 0},
      {0, 1, 0, 0, 4},
      {5, 2, 0, 0, 2},
      {0, 0, 4, 2, 0}
  };

  int source = 0;

  dijkstra(graph, source);

  return 0;
}
```

Output:

```
Vertex   Shortest Distance from Source
0        0
1        7
2        8
3        5
4        7
```